Week 2

Nodes, Topics, and Packages

Jaerock Kwon, PhD
Associate Professor
Electrical and Computer Engineering
University of Michigan-Dearborn

Lab 1 debrief

The lesson worth keeping:

- 2 topics instead of 21
- Exit code **0**, stderr **empty**
- The simulator running perfectly the whole time

> *When something is **invisible** rather than **broken**,*
> *suspect the environment before you suspect your code.*

Today you write code. Next week that habit gets tested again.

1. Workspaces and packages

Workspace anatomy

```
ros2_ws/
├── src/        your packages, and cloned upstream sources
├── build/      intermediate artifacts
├── install/    what you actually source
└── log/        build and test logs
```

- `colcon build` walks `src/`, resolves order from each `package.xml`
- Sourcing `install/setup.bash` puts those packages on your path
- `--symlink-install` links rather than copies — **edit Python without rebuilding**

Two build types

	Use for
`ament_cmake`	C++, and packages that mainly install files
`ament_python`	Pure-Python nodes

Being able to say **why** you picked one is the point.

Seven MITT packages. Zero nodes.

`mitt_bringup` · `mitt_control` · `mitt_description` · `mitt_localization`
`mitt_msgs` · `mitt_navigation` · `mitt_sim`

Launch files, YAML, URDF, message definitions, a world.

Not one C++ or Python node in the entire workspace.

Not an oversight — it is where the project is.

Today **you write the first node this repository has ever had.**

2. The shape of a node

Publisher, subscriber, timer

```
class Thing(Node):
    def __init__(self):
        super().__init__('thing')
        self.pub = self.create_publisher(Msg, '/out', 10)
        self.create_subscription(Msg, '/in', self.on_in, 10)
        self.create_timer(0.1, self.tick)     # 10 Hz
```

- The `10` is **queue depth** — part of the QoS profile (Week 3)
- **Callbacks must not block.** A slow callback stalls every other callback in that executor — and it looks like a performance problem, not a structural one

3. Rates are not a preference

Choosing a publish rate

Bounded **below** by the consumer's timeout, **above** by what it can absorb.

Constraint	Implies
`reference_timeout: 0.5 s`	**> 2 Hz** — absolute floor
Controller runs at 100 Hz	Faster than that buys nothing
≤ 100 ms p95 latency budget	≥ 20 Hz, so one drop does not blow it

10–20 Hz is the sensible band.

Lab 2 has you publish at 10 Hz, then at 1 Hz — and watch the floor enforce itself.

4. A car is not a differential-drive robot

The constraint you cannot design around

$$\omega = \frac{v \tan \delta}{L}$$

Yaw rate is **proportional to forward speed**.

At `v = 0`, ω is zero **for any steering angle whatsoever**.

A car cannot pirouette.

Two consequences, one today and one in November

- **Today:** your square driver must keep `linear.x` non-zero **through the turn**.
  Set it to zero and the vehicle stops turning entirely.
- **In November:** the navigation stack needs a planner that can plan **reversing manoeuvres** — because for this vehicle, "turn around" is not a primitive.

Formal treatment in Week 5. Feel it first, in the lab.

5. Where odometry comes from

One shaft, of two motors

Your node subscribes to `/ackermann_steering_controller/odometry` and reads a position.

On the **real** vehicle:

- Both rear motors wired **in parallel** to a single driver channel
- Only **one** carries an encoder
- The measurement is taken at **one motor shaft**

What that shaft cannot see

Blind to	Because
Gearbox backlash	It is measured **upstream** of the gearbox
Tyre slip and deformation	Wheel rotation ≠ ground distance
The other rear wheel	In a turn the two travel different distances

This is a documented, deliberately accepted design decision — ADR C — not an oversight.

Which is why odometry is fused, not trusted.

The EKF in Week 6 is a response to a specific, named measurement gap — not a formality bolted on because textbooks have one.

One more thing the record will not let you assume

The encoder's **pulses per revolution is unverified.**

- The predecessor project's firmware pins **52 PPR**
- Its own bill of materials lists a **16 PPR** motor
- Nobody caught it, because in simulation nothing complains

Finding F7 forbids inheriting either number. Measure it.

(The roll-out calibration bypasses PPR entirely — which is exactly why it is done that way.)

Lab 2 — Drive a Square

1. Create a package with `ros2 pkg create`
2. Publish a square trajectory, open loop, at 10 Hz
3. Subscribe to odometry; measure the drift
4. Explain why the square does not close — **two distinct reasons**
5. **Break it on purpose** — publish at 1 Hz and find the timeout

Deliverables: package · odometry log after 1 and 3 squares · written answers · AI declaration

Due before Week 3 (9/28)

Next week

Services, parameters, and QoS — and the second silent failure.

Reading: **Zero to Robot** ch. 3 and ch. 19, plus safety.md §1.

Chuseok is 9/24–26. There is no substitute holiday, so **Week 4 meets normally on 10/05** — but you get a longer stretch for Lab 3. Use it if your setup is still fighting you.